Session 5: Python Modules and Pandas

In [1]:
```
## Loading and Exploring Data
```

In [2]:
```
import pandas
%pwd
%cd mukh11
```

```
/home/mukh11/mukh11
```

In [3]:
```
station_ids = pandas.read_csv("data/Niederschlag_1981-2010_Stationsliste.
```

In [4]:
```
station_ids
```

Out[4]:

	Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	B
0	1	Aach	47.841299	8.849299	478.0	Wi
1	2	Aachen (Kläranlage)	50.806581	6.099638	138.0	
2	3	Aachen	50.782659	6.094087	202.0	
3	4	Aachen-Brand	50.768313	6.120705	243.0	
4	6	Aalen-Unterrombach	48.836072	10.059764	455.0	Wi
...	...	...	...	...	...	
5735	19697	Wassertrüdingen/Wörnitz	49.046341	10.602684	435.0	
5736	19698	Werda/Vogtland	50.448020	12.305168	595.0	
5737	19701	Wertingen/Zusam	48.557500	10.674167	430.0	
5738	19706	Oberndorf am Neckar	48.285077	8.576395	516.0	Wi
5739	19781	Ingolstadt	48.742909	11.423324	367.0	

5740 rows × 7 columns

In [5]:
```
station_ids.columns
```

Out[5]:
```
Index(['Stations_id', 'Stationsname', 'geogr. Breite', 'geogr. Laenge',
       'Stationshoehe', 'Bundesland', 'Unnamed: 6'],
      dtype='str')
```

In [6]:
```
station_ids.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5740 entries, 0 to 5739
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Stations_id     5740 non-null   int64
 1   Stationsname    5740 non-null   str
 2   geogr. Breite   5740 non-null   float64
 3   geogr. Laenge   5740 non-null   float64
 4   Stationshoehe   5740 non-null   float64
 5   Bundesland      5740 non-null   str
 6   Unnamed: 6      0 non-null      float64
dtypes: float64(4), int64(1), str(2)
memory usage: 314.0 KB
```

In [7]:
```
station_ids.head()
```

Out[7]:

	Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland	U
0	1	Aach	47.841299	8.849299	478.0	Baden-Württemberg	
1	2	Aachen (Kläranlage)	50.806581	6.099638	138.0	Nordrhein-Westfalen	
2	3	Aachen	50.782659	6.094087	202.0	Nordrhein-Westfalen	
3	4	Aachen-Brand	50.768313	6.120705	243.0	Nordrhein-Westfalen	
4	6	Aalen-Unterrombach	48.836072	10.059764	455.0	Baden-Württemberg	

In [8]:
```
station_ids.tail()
```

Out[8]:

	Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	B
5735	19697	Wassertrüdingen/Wörnitz	49.046341	10.602684	435.0	
5736	19698	Werda/Vogtland	50.448020	12.305168	595.0	
5737	19701	Wertingen/Zusam	48.557500	10.674167	430.0	
5738	19706	Oberndorf am Neckar	48.285077	8.576395	516.0	Wü
5739	19781	Ingolstadt	48.742909	11.423324	367.0	

In [9]:
```
station_ids.columns
```

Out[9]:
```
Index(['Stations_id', 'Stationsname', 'geogr. Breite', 'geogr. Laenge',
       'Stationshoehe', 'Bundesland', 'Unnamed: 6'],
      dtype='str')
```

In [10]:
```
station_ids.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5740 entries, 0 to 5739
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Stations_id     5740 non-null   int64
 1   Stationsname    5740 non-null   str
 2   geogr. Breite   5740 non-null   float64
 3   geogr. Laenge   5740 non-null   float64
 4   Stationshoehe   5740 non-null   float64
 5   Bundesland      5740 non-null   str
 6   Unnamed: 6      0 non-null      float64
dtypes: float64(4), int64(1), str(2)
memory usage: 314.0 KB
```

In [11]:
```
station_ids.head()
```

Out[11]:

	Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland	U
0	1	Aach	47.841299	8.849299	478.0	Baden-Württemberg	
1	2	Aachen (Kläranlage)	50.806581	6.099638	138.0	Nordrhein-Westfalen	
2	3	Aachen	50.782659	6.094087	202.0	Nordrhein-Westfalen	
3	4	Aachen-Brand	50.768313	6.120705	243.0	Nordrhein-Westfalen	
4	6	Aalen-Unterrombach	48.836072	10.059764	455.0	Baden-Württemberg	

In [12]:
```
station_ids.columns
```

Out[12]:
```
Index(['Stations_id', 'Stationsname', 'geogr. Breite', 'geogr. Laenge',
       'Stationshoehe', 'Bundesland', 'Unnamed: 6'],
      dtype='str')
```

In [13]:
```
station_ids.drop(columns=["Unnamed: 6"], inplace=True)
```

In [14]:
```
station_ids = station_ids.set_index("Stations_id")
```

In [15]:
```
station_ids.columns
```

Out[15]:
```
Index(['Stationsname', 'geogr. Breite', 'geogr. Laenge', 'Stationshoeh
e',
       'Bundesland'],
      dtype='str')
```

In [16]:
```
station_ids["Bundesland"]
```

Out[16]:
```
Stations_id
1          Baden-Württemberg
2        Nordrhein-Westfalen
3        Nordrhein-Westfalen
4        Nordrhein-Westfalen
6          Baden-Württemberg
                 ...
19697                Bayern
19698               Sachsen
19701                Bayern
19706      Baden-Württemberg
19781                Bayern
Name: Bundesland, Length: 5740, dtype: str
```

In [17]:
```
station_ids.groupby("Bundesland").count().sort_values("Stationsname", asc
```

Out[17]:

Bundesland	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe
Bayern	1337	1337	1337	1337
Baden-Württemberg	619	619	619	619
Niedersachsen	611	611	611	611
Hessen	436	436	436	436
Nordrhein-Westfalen	436	436	436	436
Sachsen-Anhalt	411	411	411	411
Thüringen	391	391	391	391
Sachsen	356	356	356	356
Brandenburg	302	302	302	302
Rheinland-Pfalz	264	264	264	264
Mecklenburg-Vorpommern	237	237	237	237
Schleswig-Holstein	223	223	223	223
Saarland	54	54	54	54
Berlin	29	29	29	29
Hamburg	19	19	19	19
Bremen	15	15	15	15

In [18]:
```
mask = station_ids.Bundesland == "Bayern"
```

In [19]:
```
mask
```

Out[19]:
```
Stations_id
1        False
2        False
3        False
4        False
6        False
         ...
19697     True
19698    False
19701     True
19706    False
19781     True
Name: Bundesland, Length: 5740, dtype: bool
```

In [20]:
```
station_ids[mask]
```

Out[20]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bund
10	Abbach,Bad-Dünzling	48.885697	12.107925	378.0	
15	Abenberg	49.234641	10.966676	391.5	
16	Abenberg-Wassermungenau	49.219251	10.884495	370.0	
17	Abensberg-Sandharlanden	48.842100	11.819366	380.0	
24	Achthal	47.833300	12.783300	582.0	
...	...	...	...	...	
19694	Tittmoning/Oberbayern	48.066895	12.763446	382.0	
19695	Untergriesbach/Niederbayern	48.599508	13.646542	495.0	
19697	Wassertrüdingen/Wörnitz	49.046341	10.602684	435.0	
19701	Wertingen/Zusam	48.557500	10.674167	430.0	
19781	Ingolstadt	48.742909	11.423324	367.0	

1337 rows × 5 columns

In [21]:
```
bayern = station_ids[mask]
```

In [22]:
```
station_ids.Bundesland.iloc[-1]
```

Out[22]:
```
'Bayern'
```

In [23]:
```
bayern.loc[17]
```

Out[23]:
```
Stationsname     Abensberg-Sandharlanden
geogr. Breite                    48.8421
geogr. Laenge                  11.819366
Stationshoehe                      380.0
Bundesland                       Bayern
Name: 17, dtype: object
```

In [24]:
```
mask = station_ids.Bundesland == "Berlin"
```

In [25]:
```
berlin = station_ids[mask]
berlin
```

Out[25]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
394	Berlin (Rosenthal)	52.590285	13.383258	43.0	Berlin
399	Berlin-Alexanderplatz	52.519785	13.405678	35.8	Berlin
400	Berlin-Buch	52.630956	13.502135	60.0	Berlin
401	Berlin-Charlottenburg	52.516700	13.266700	55.0	Berlin
402	Berlin-Dahlem (LFAG)	52.456399	13.299675	55.0	Berlin
403	Berlin-Dahlem (FU)	52.453711	13.301731	51.0	Berlin
404	Berlin-Friedrichsfelde	52.504696	13.521539	38.0	Berlin
406	Berlin-Friedrichshain	52.516700	13.416700	33.0	Berlin
407	Berlin-Frohnau	52.646077	13.285772	48.0	Berlin
409	Berlin-Johannisthal	52.450000	13.500000	35.0	Berlin
410	Berlin-Kaniswall	52.404034	13.730854	33.0	Berlin
412	Berlin-Karlshorst	52.483300	13.516700	35.0	Berlin
413	Berlin-Kaulsdorf	52.504396	13.579930	39.0	Berlin
414	Berlin-Köpenick	52.475300	13.589930	36.0	Berlin
416	Berlin-Lichtenrade	52.406906	13.412659	47.0	Berlin
417	Berlin-Lichterfelde (Süd)	52.414404	13.303575	43.0	Berlin
418	Berlin-Lichterfelde Ost	52.408905	13.331571	45.0	Berlin
419	Berlin-Malchow	52.575287	13.483243	53.0	Berlin
420	Berlin-Marzahn	52.544744	13.559789	61.0	Berlin
421	Berlin-Marzahn, Apw	52.533300	13.550000	52.0	Berlin
422	Berlin-Mitte	52.533300	13.383300	35.0	Berlin
425	Berlin-Rudow	52.404407	13.483848	38.0	Berlin

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
426	Berlin-Schmöckwitz	52.382302	13.623638	36.0	Berlin
429	Berlin-Spandau	52.566685	13.169392	32.0	Berlin
430	Berlin-Tegel	52.564402	13.308805	35.5	Berlin
432	Berlin-Tegeler Fliesstal	52.604382	13.295172	36.0	Berlin
433	Berlin-Tempelhof	52.467451	13.402120	47.7	Berlin
434	Berlin-Treptow	52.455301	13.469949	35.0	Berlin
435	Berlin-Zehlendorf	52.428902	13.232686	45.0	Berlin

In [26]:
```
berlin["Stationsname"]
```

Out[26]:
```
Stations_id
394           Berlin (Rosenthal)
399        Berlin-Alexanderplatz
400                  Berlin-Buch
401        Berlin-Charlottenburg
402         Berlin-Dahlem (LFAG)
403           Berlin-Dahlem (FU)
404       Berlin-Friedrichsfelde
406        Berlin-Friedrichshain
407              Berlin-Frohnau
409          Berlin-Johannisthal
410             Berlin-Kaniswall
412            Berlin-Karlshorst
413             Berlin-Kaulsdorf
414              Berlin-Köpenick
416           Berlin-Lichtenrade
417    Berlin-Lichterfelde (Süd)
418      Berlin-Lichterfelde Ost
419              Berlin-Malchow
420              Berlin-Marzahn
421          Berlin-Marzahn, Apw
422                 Berlin-Mitte
425                 Berlin-Rudow
426           Berlin-Schmöckwitz
429               Berlin-Spandau
430                  Berlin-Tegel
432     Berlin-Tegeler Fliesstal
433              Berlin-Tempelhof
434                Berlin-Treptow
435            Berlin-Zehlendorf
Name: Stationsname, dtype: str
```

In [27]:
```
berlin[berlin["Stationsname"].str.contains("Alexander")]
```

Out[27]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
399	Berlin-Alexanderplatz	52.519785	13.405678	35.8	Berlin

In [28]:
```
precip = pandas.read_csv(
    "data/produkt_precipitation_399_akt.txt",
    delimiter=" *; *",
    engine="python"
)
```

In [29]:
```
precip.head()
precip.columns
```

Out[29]:
```
Index(['STATIONS_ID', 'MESS_DATUM', 'QUALITAETS_NIVEAU', 'STRUKTUR_VERSI
ON',
       'NIEDERSCHLAGSHOEHE', 'eor'],
      dtype='str')
```

In [30]:
```
precip.MESS_DATUM = pandas.to_datetime(
    precip.MESS_DATUM,
    format="%Y%m%d%H"
)
```

In [31]:
```
precip = precip[["MESS_DATUM", "NIEDERSCHLAGSHOEHE"]]

precip = precip.rename(columns={
    "MESS_DATUM": "date",
    "NIEDERSCHLAGSHOEHE": "precipitation"
})

precip = precip.set_index("date")
```

In [32]:
```
monthly = precip.resample("ME").sum()
```

In [33]:
```
monthly.head()
monthly.describe()
```

Out[33]:

	precipitation
count	129.000000
mean	39.528372
std	28.812164
min	0.000000
25%	22.910000
50%	33.640000
75%	48.600000
max	166.300000

In [34]:
```
precip_ref = pandas.read_csv(
    "data/Niederschlag_1981-2010.txt",
    delimiter=" *; *",
    encoding="iso-8859-1",
    engine="python",
    decimal=","
)
```

In [35]:
```
berlin[berlin["Stationsname"].str.contains("Alexander", case=False)]
```

Out[35]:

Stations_id	Stationsname	geogr. Breite	geogr. Laenge	Stationshoehe	Bundesland
399	Berlin-Alexanderplatz	52.519785	13.405678	35.8	Berlin

In [36]:
```
station_id = 399
```

In [37]:
```
precip_ref.head()
```

Out[37]:

	Stations_id	Bezugszeitraum	Datenquelle	Jan.	Feb.	März	Apr.	Mai	Jun.	Ju
0	1	1981-2010	46	48.4	45.6	49.7	51.5	88.7	93.0	95.
1	2	1981-2010	45	74.3	66.0	64.4	58.2	70.9	79.8	71.
2	3	1981-2010	45	68.1	63.6	67.0	55.7	72.0	80.3	75.
3	4	1981-2010	46	68.9	66.1	67.4	55.9	71.5	81.2	75.
4	6	1981-2010	45	66.2	65.3	75.2	55.8	93.0	83.0	95.

In [38]:
```
precip_ref.columns
```

Out[38]:
```
Index(['Stations_id', 'Bezugszeitraum', 'Datenquelle', 'Jan.', 'Feb.',
'März',
       'Apr.', 'Mai', 'Jun.', 'Jul.', 'Aug.', 'Sept.', 'Okt.', 'Nov.',
'Dez.',
       'Jahr', 'Unnamed: 16'],
      dtype='str')
```

In [39]:
```
precip_ref.loc[station_id]
```

Out[39]:
```
Stations_id             462
Bezugszeitraum    1981-2010
Datenquelle              45
Jan.                   90.7
Feb.                   68.5
März                   79.5
Apr.                   52.2
Mai                    64.1
Jun.                   95.4
Jul.                   96.7
Aug.                   89.8
Sept.                  97.9
Okt.                  105.9
Nov.                   92.7
Dez.                   98.0
Jahr                 1031.2
Unnamed: 16             NaN
Name: 399, dtype: object
```

In [49]:
```
precip_ref
```

Out[49]:

	Stations_id	Bezugszeitraum	Datenquelle	Jan.	Feb.	März	Apr.	Mai	Jun.
0	1	1981-2010	46	48.4	45.6	49.7	51.5	88.7	93.0
1	2	1981-2010	45	74.3	66.0	64.4	58.2	70.9	79.8
2	3	1981-2010	45	68.1	63.6	67.0	55.7	72.0	80.3
3	4	1981-2010	46	68.9	66.1	67.4	55.9	71.5	81.2
4	6	1981-2010	45	66.2	65.3	75.2	55.8	93.0	83.0
...	...	...	...	...	...	...	...	...	...
5704	19697	1981-2010	45	54.4	48.5	56.9	42.7	73.6	76.4
5705	19698	1981-2010	45	59.0	54.4	63.9	57.0	75.8	86.0
5706	19701	1981-2010	45	46.1	39.8	54.0	52.0	84.1	85.8
5707	19706	1981-2010	45	89.0	82.0	82.3	69.9	102.4	84.0
5708	19781	1981-2010	45	47.6	40.1	47.6	42.0	75.1	77.9

5709 rows × 17 columns

In [51]:
```
precip_ref.drop(precip_ref.columns[-1],axis=1, inplace=True)
```

In [52]:
```
precip_ref
```

Out[52]:

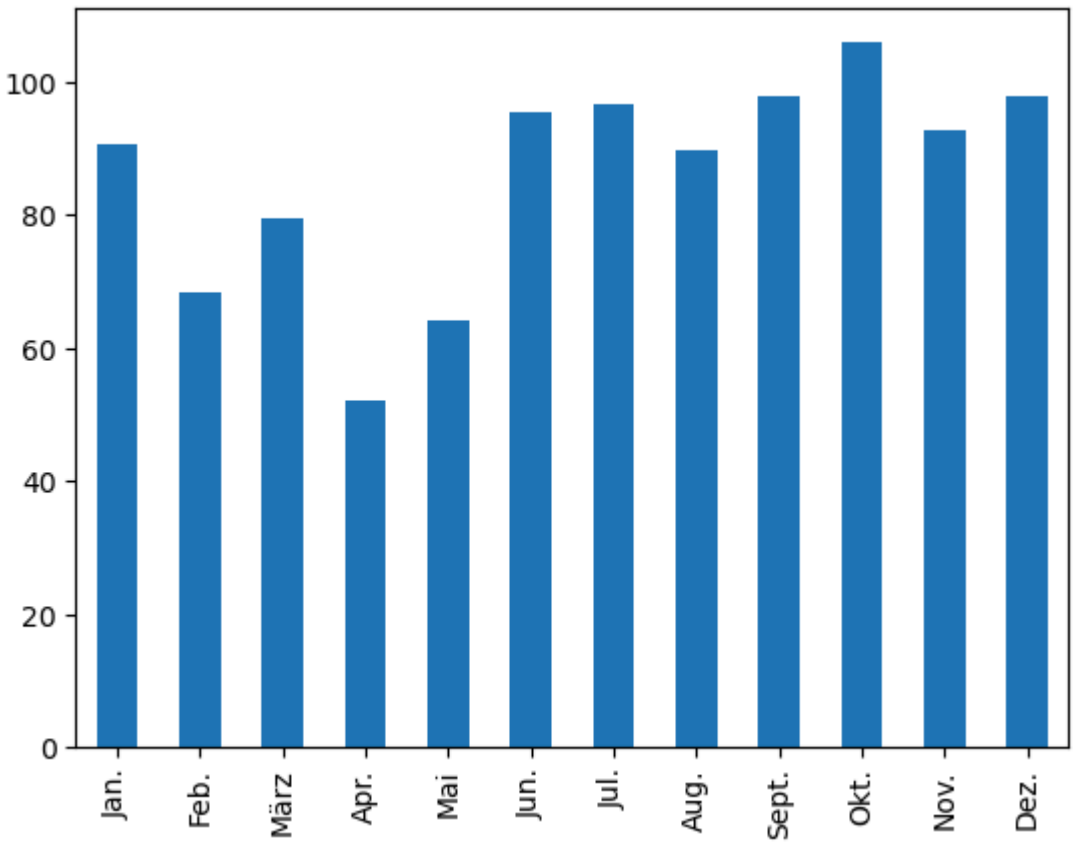
	Stations_id	Bezugszeitraum	Datenquelle	Jan.	Feb.	März	Apr.	Mai	Jun.
0	1	1981-2010	46	48.4	45.6	49.7	51.5	88.7	93.0
1	2	1981-2010	45	74.3	66.0	64.4	58.2	70.9	79.8
2	3	1981-2010	45	68.1	63.6	67.0	55.7	72.0	80.3
3	4	1981-2010	46	68.9	66.1	67.4	55.9	71.5	81.2
4	6	1981-2010	45	66.2	65.3	75.2	55.8	93.0	83.0
...	...	...	...	...	...	...	...	...	...
5704	19697	1981-2010	45	54.4	48.5	56.9	42.7	73.6	76.4
5705	19698	1981-2010	45	59.0	54.4	63.9	57.0	75.8	86.0
5706	19701	1981-2010	45	46.1	39.8	54.0	52.0	84.1	85.8
5707	19706	1981-2010	45	89.0	82.0	82.3	69.9	102.4	84.0
5708	19781	1981-2010	45	47.6	40.1	47.6	42.0	75.1	77.9

5709 rows × 16 columns

In [57]:
```
precip_ref=precip_ref.loc[:, "Jan.":"Dez."]
```

In [58]:
```
precip_ref.loc[station_id].plot.bar()
```

Out[58]:
```
<Axes: >
```



In [59]:
```
precip = pandas.read_csv(
    "data/produkt_precipitation_399_akt.txt",
```

```
    delimiter=" *; *",
    engine="python"
)
```

In [60]:
```
precip.head()
precip.columns
```

Out[60]:
```
Index(['STATIONS_ID', 'MESS_DATUM', 'QUALITAETS_NIVEAU', 'STRUKTUR_VERSI
ON',
       'NIEDERSCHLAGSHOEHE', 'eor'],
      dtype='str')
```

In [61]:
```
precip["MESS_DATUM"] = pandas.to_datetime(
    precip["MESS_DATUM"],
    format="%Y%m%d%H"
)
```

In [62]:
```
precip.head()
```

Out[62]:

	STATIONS_ID	MESS_DATUM	QUALITAETS_NIVEAU	STRUKTUR_VERSION	NIEDERSC
0	399	2015-08-20 01:00:00	3	0	
1	399	2015-08-20 02:00:00	3	0	
2	399	2015-08-20 03:00:00	3	0	
3	399	2015-08-20 04:00:00	3	0	
4	399	2015-08-20 05:00:00	3	0	

In [63]:
```
precip = precip[["MESS_DATUM", "NIEDERSCHLAGSHOEHE"]]
```

In [64]:
```
precip = precip.rename(columns={
    "MESS_DATUM": "date",
    "NIEDERSCHLAGSHOEHE": "precipitation"
})
```

In [65]:
```
precip = precip.set_index("date")
```

In [66]:
```
monthly = precip.resample("ME").sum()
```

In [67]:
```
monthly.head()
```

Out[67]:

date	precipitation
2015-08-31	18.90
2015-09-30	29.19
2015-10-31	60.91
2015-11-30	58.59
2015-12-31	27.34

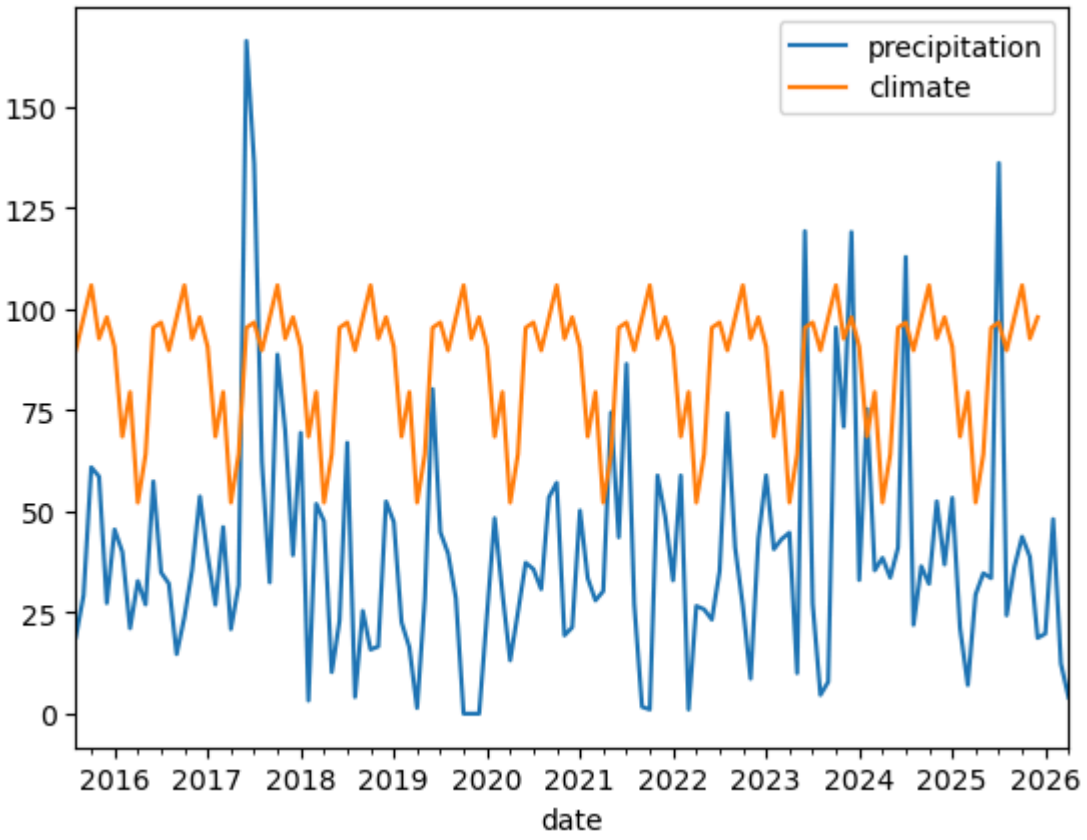
In [68]:
```
refs = pandas.DataFrame(
    {"precipitation": precip_ref.loc[station_id].to_list() * 11},
    index=pandas.date_range("2015-01-01", "2025-12-31", freq="ME")
)
```

In [69]:
```
monthly["climate"] = refs
```

In [70]:
```
monthly["anomaly"] = monthly["precipitation"] - monthly["climate"]
```

In [71]:
```
monthly[["precipitation", "climate"]].plot()
```

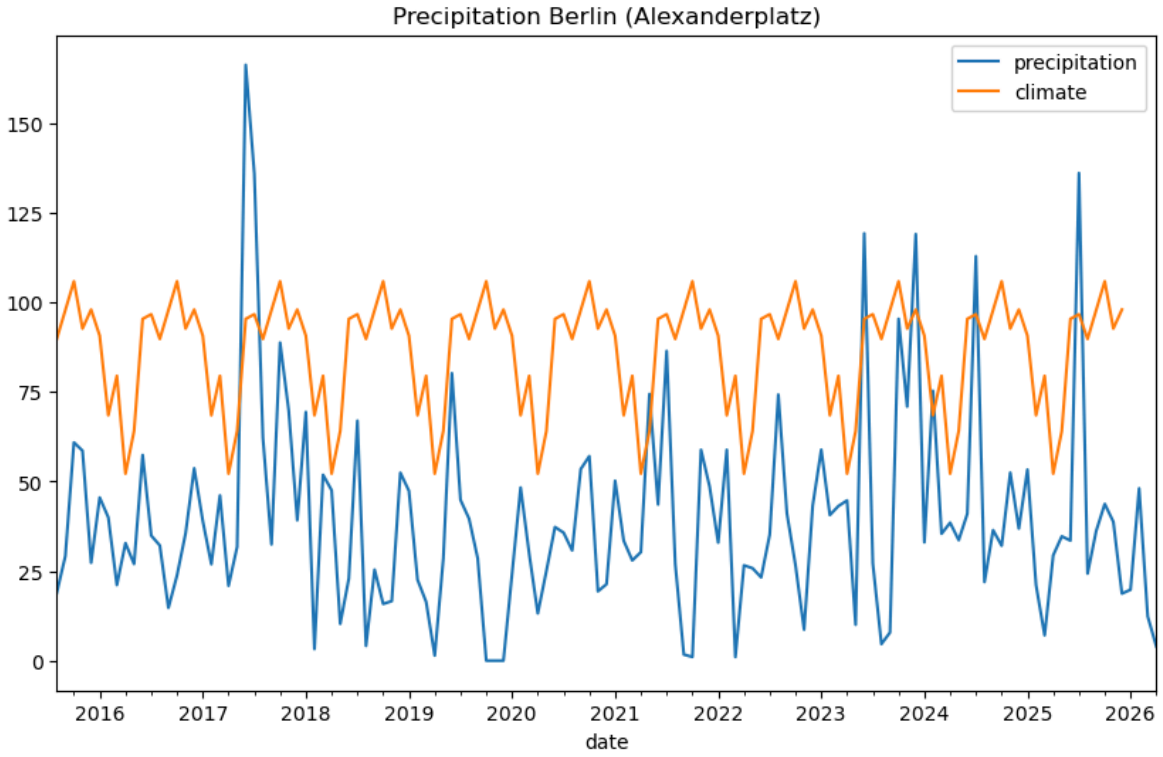
Out[71]: <Axes: xlabel='date'>



In [73]:
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10,6))

monthly[["precipitation", "climate"]].plot(ax=ax)
```

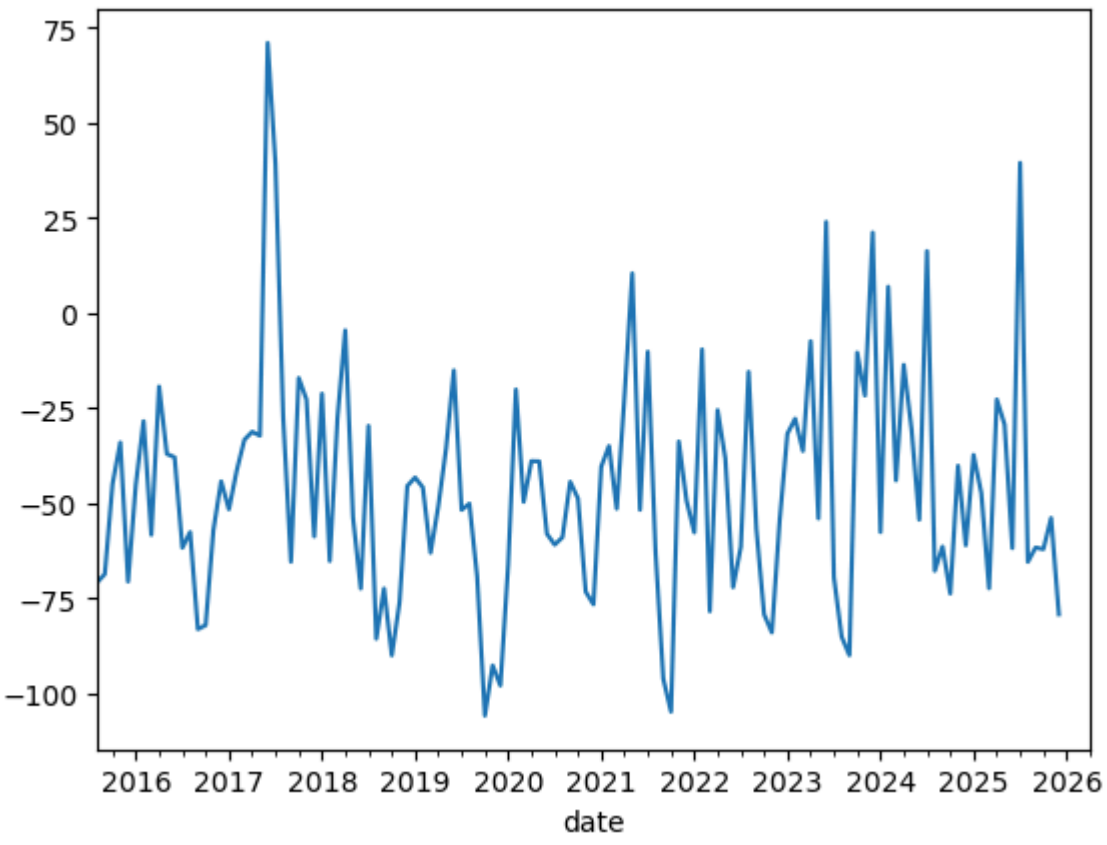
```
ax.set_title("Precipitation Berlin (Alexanderplatz)")
```

Out[73]: Text(0.5, 1.0, 'Precipitation Berlin (Alexanderplatz)')



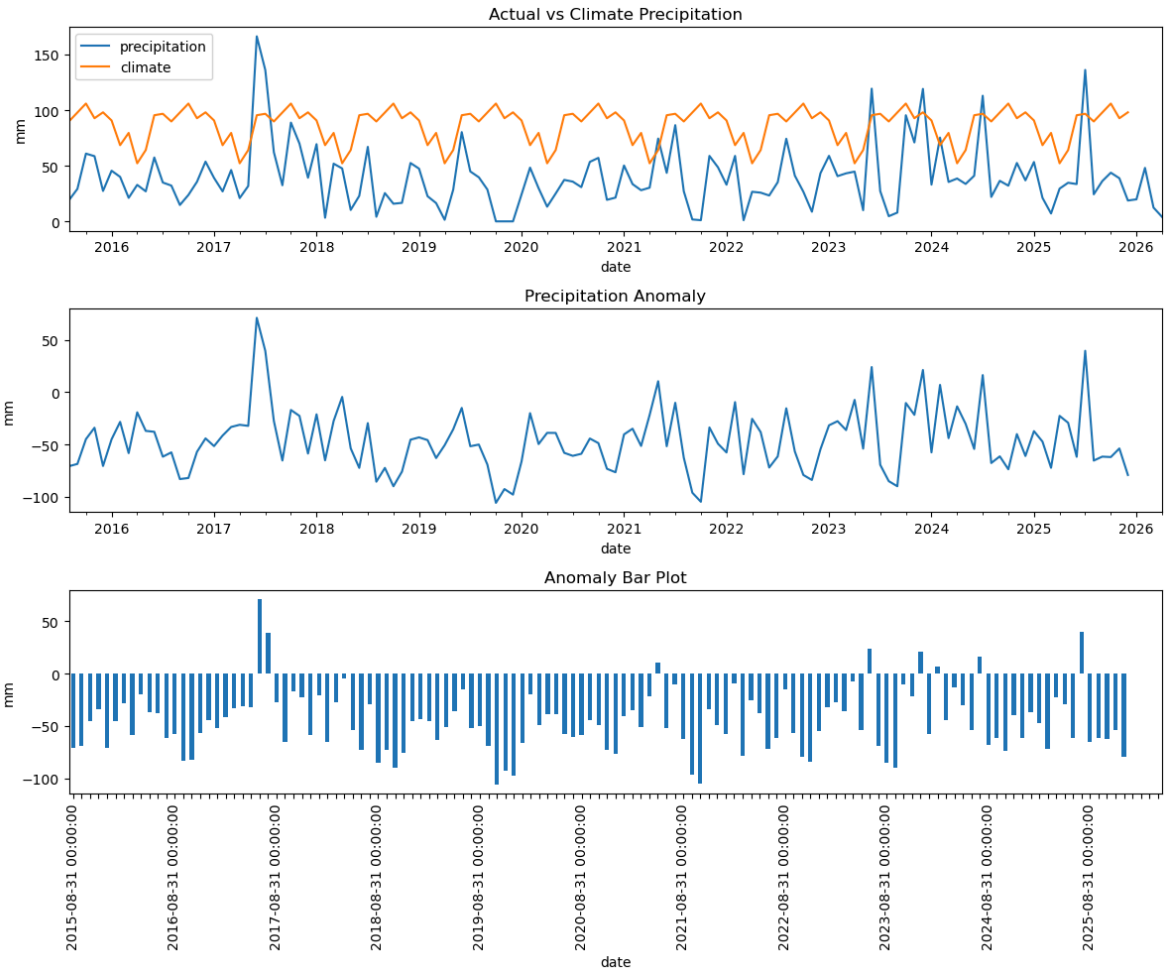
In [74]:
```
monthly["anomaly"].plot()
```

Out[74]: <Axes: xlabel='date'>



In [78]:
```
import matplotlib.pyplot as plt
```

```
fig, axs = plt.subplots(3, 1, figsize=(12,10))

# Actual vs Climate
monthly[["precipitation", "climate"]].plot(ax=axs[0])
axs[0].set_title("Actual vs Climate Precipitation")
axs[0].set_ylabel("mm")

# Anomaly line plot
monthly["anomaly"].plot(ax=axs[1])
axs[1].set_title("Precipitation Anomaly")
axs[1].set_ylabel("mm")

# Anomaly bar plot
monthly["anomaly"].plot(kind="bar", ax=axs[2])

for i, label in enumerate(axs[2].get_xticklabels()):
    if i % 12 != 0:
        label.set_visible(False)

axs[2].set_title("Anomaly Bar Plot")
axs[2].set_ylabel("mm")

# show only yearly labels
ax.set_xticks(range(0, len(monthly), 12))
ax.set_xticklabels(monthly.index.strftime("%Y")[::12], rotation=45)

axs[2].set_title("Anomaly Bar Plot")
axs[2].set_ylabel("mm")

plt.tight_layout()
```



In [ ]: